Département Informatique
Promotion 2022-2025

ScanX

Rapport technique individuel de

Thibault LEMERCIER

Projet effectué au département informatique de l'IUT de Metz

Introduction

Le projet ambitieux de création d'une application mobile visant à utiliser la caméra de l'appareil pour scanner la carte d'identité d'un client représente une avancée significative dans le domaine de la sécurité et de la prévention. L'objectif principal est d'extraire les informations personnelles des bandes MRZ (Machine Readable Zone) présentes sur la carte d'identité. Ces données seront ensuite comparées avec l'API du gel des avoirs du site gouvernemental français (https://gels-avoirs.dgtresor.gouv.fr/) dans le but de déterminer si le client est interdit en France. En cas de restriction, l'application affichera la raison correspondante, permettant ainsi de filtrer les entrées dans des lieux sensibles tels que les casinos.

Point Technique : Décryptage des Bandes MRZ

La section suivante détaillera mon implication technique dans le processus de décryptage des bandes MRZ. Cette étape critique du projet constitue le fondement de la collecte des informations nécessaires à la vérification du statut du client vis-à-vis de la législation française.

Ce point technique implique plusieurs sous-tâches :

- Acquisition des données depuis la caméra.
    - Accès à la fonctionnalité caméra de l’appareil.
    - Envoie de l’image capturé en requête POST vers un serveur PHP.

- Traitement des images.
    - Récupération de l’image sur le serveur PHP.
    - Traitement de l’image avec Tesseract OCR.

- Algorithme de décodage des bandes MRZ.
    - Recherche sur les informations contenues des bandes MRZ.
    - Création de l’algorithmes de décodage des bandes.
    - Mise en base de données des informations utiles.

- Intégration avec l'API gouvernementale.
    - Traitement des données de l’API.
    - Mise en base de données des informations utiles.
    - Comparaison de la table client et de l’API gouvernementale.

La section suivante abordera en détail les technologies, langages, et les défis rencontrés tout au long de ce processus de développement.

Développement

Technologies et Langages Utilisés

React Native :

- Description : React Native a été choisi comme framework de développement pour l'application mobile. Cette technologie permet une approche de développement multiplateforme, ce qui signifie que le code peut être réutilisé pour créer des applications à la fois sur Android et iOS, réduisant ainsi le temps de développement.

- Utilisation : La structure de l'interface utilisateur, les composants, et la logique de l'application ont été développées en utilisant React Native, offrant ainsi une expérience utilisateur cohérente sur différentes plateformes.

TypeScript :

- Description : TypeScript, en tant que sur ensemble de JavaScript, a été intégré dans le développement de l'application pour apporter un typage statique au code JavaScript. Cela améliore la robustesse du code en détectant les erreurs potentielles pendant le processus de développement.

- Utilisation : Toute la partie frontend de l’application a été écrite en TypeScript pour bénéficier des avantages de la vérification statique des types.

PHP :

- Description : PHP a été utilisé du côté serveur pour gérer le traitement des images capturées par la caméra de l'appareil mais également dans la gestion de l’API gouvernementale. PHP est adapté au traitement de requêtes HTTP et a été utilisé pour recevoir les images, les traiter avec Tesseract OCR, et stocker les données pertinentes dans une base de données.

- Utilisation : Le serveur PHP a été mis en place pour recevoir les requêtes POST contenant les images capturées. Il a ensuite orchestré le traitement de l'image en utilisant Tesseract OCR et a stocké les informations extraites dans une base de données afin de les comparer avec les données de l’API gouvernementale.

Tesseract OCR :

- Description : Tesseract OCR a été intégré pour le traitement d'image et l'OCR (Optical Character Recognition). Cet outil open source permet de reconnaître et d'extraire du texte à partir d'images.

- Utilisation : Dans la phase de traitement d'image, Tesseract OCR a été utilisé pour extraire les informations textuelles des bandes MRZ des images capturées par la caméra.

Analyse

La phase d'analyse du décryptage des bandes MRZ revêt une importance particulière, étant donné la diversité des documents d'identité tels que les cartes d'identité, les permis de conduire et les passeports. L'intégration de Tesseract OCR dans le processus est une composante clé, permettant la capture précise de la MRZ à partir de photos.

Carte d'Identité :

- Les cartes d'identité peuvent présenter une variété de formats MRZ, incluant des champs tels que le nom, le prénom, la date de naissance, et le numéro de carte d'identité. La flexibilité dans l'analyse de la MRZ est essentielle pour s'adapter aux variations nationales dans la structure des cartes d'identité. ci-dessous.

ci-dessous

Permis de Conduire :

- Les MRZ des permis de conduire peuvent contenir des informations comme le nom, la date de naissance et le numéro de permis, avec des variations entre les régions. L'utilisation de Tesseract OCR doit être optimisée pour extraire avec précision ces informations, même dans des formats et des langues différents.

Passeport :

- Les passeports suivent les normes internationales de l'OACI pour la MRZ, contenant des informations standardisées telles que le nom, le numéro de passeport et la date de naissance. L'analyse doit respecter ces normes pour une reconnaissance précise.

Intégration de Tesseract OCR :

- Capture de MRZ via Photo : L'utilisation de Tesseract OCR permet la capture de la MRZ à partir de photos prises par la caméra de l'appareil. Cette intégration doit être optimisée pour garantir la précision de la reconnaissance de texte, même dans des conditions variées.

L'analyse approfondie de ces spécificités guide le développement d'un processus de décryptage robuste. L'intégration de Tesseract OCR joue un rôle central dans la capture précise de la MRZ à partir de photos, fournissant ainsi une base technique solide pour le développement de l'application mobile.

Conception

La conception du décryptage des MRZ en TypeScript pour la gestion locale et PHP pour le traitement sur le serveur a nécessité une approche flexible en raison des défis rencontrés lors de l'intégration de packages Composer initialement envisagés en PHP.

Utilisation de TypeScript pour la Gestion Locale :

- Librairie "mrz" : La librairie "mrz" en TypeScript est employée pour le décryptage local des MRZ directement sur l'appareil de l'utilisateur, offrant réactivité et respect de la confidentialité.

- Gestion des Types de Documents : TypeScript facilite la gestion des différents formats MRZ pour les cartes d'identité, les permis de conduire et les passeports, en s'adaptant aux spécificités de la librairie "mrz".

- Intégration Transparente : Le décryptage en TypeScript s'intègre naturellement à l'application mobile, fournissant une expérience utilisateur fluide lors de la capture des images des documents d'identité.

Décryptage en PHP avec Algorithme Personnalisé :

- Problèmes Rencontrés avec les Packages Composer : Les packages Composer envisagés pour PHP n'ont pas répondu de manière adéquate aux besoins spécifiques, présentant des erreurs de décodage et une incompatibilité avec les formats MRZ.

- Étude du Décryptage des Bandes MRZ : Face à ces défis, une étude approfondie du décodage des MRZ a été entreprise, conduisant au développement d'un algorithme personnalisé pour surmonter ces limitations.

- Intégration avec l'Application et Stockage des Données : Les images capturées sont envoyées au serveur PHP, où l'algorithme personnalisé est appliqué pour extraire les informations, qui sont ensuite stockées de manière sécurisée dans une base de données.

Réalisation

La mise en œuvre de la partie locale du décryptage des MRZ en TypeScript repose sur l'utilisation de la librairie "mrz".

- Utilisation de la Librairie "mrz" en TypeScript : L'image jointe démontre le code TypeScript intégrant la librairie "mrz". Cette implémentation permet le décryptage local des MRZ directement sur l'appareil de l'utilisateur, assurant ainsi une réactivité rapide. ci-dessous

- Adaptabilité aux Types de Documents : Les images illustrent également la gestion des différents types de documents d'identité, montrant comment TypeScript, avec la librairie "mrz", s'adapte aux spécificités des formats MRZ des cartes d'identité, des permis de conduire et des passeports. ci-dessous

Réalisation du Décryptage des MRZ - Partie Serveur en PHP :

La partie serveur du décryptage des MRZ repose sur un algorithme personnalisé, illustré dans une image en annexe. Cette approche sur mesure a été développée pour surmonter les limitations rencontrées avec les packages Composer initialement envisagés.

- Algorithme Personnalisé en PHP : L'image jointe expose l'algorithme personnalisé fonctionnant pour récupérer le type du document (pour l'exemple). Cette solution a été élaborée après une étude approfondie des spécificités des bandes MRZ et des problèmes rencontrés avec les packages Composer. **ci-dessous**

Cette approche divisée entre la partie locale en TypeScript avec la librairie "mrz" et la partie serveur en PHP avec un algorithme personnalisé garantit une solution complète et adaptative pour le décryptage des MRZ dans le cadre de l'application mobile.

Difficultés rencontrées

La réalisation du décryptage des MRZ a été marquée par des défis significatifs, mettant à l'épreuve ma capacité à résoudre des problèmes complexes. Les principales difficultés ont émergé lors de l'implémentation de la partie décodage en TypeScript et de son transfert vers la partie serveur en PHP.

Décodage en TypeScript Limité au Client :

- Au départ, la totalité du processus de décodage des MRZ en TypeScript était fonctionnelle, mais exclusivement côté client. Cette approche soulevait des préoccupations en termes de sécurité, car les requêtes et le décodage se déroulaient localement sur l'appareil de l'utilisateur.

Nécessité de la Partie Locale malgré l'Absence de Connexion Internet :

- Pour éviter l'inutilité de la partie locale, réalisée en TypeScript, en cas d'absence de connexion internet, une décision stratégique a été prise de maintenir cette fonctionnalité. Ainsi, même sans connexion, l'application peut continuer à effectuer des scans.

Transfert vers le Serveur en PHP et Problèmes avec les Packages :

- Afin de résoudre le problème de décodage limité au client, j'ai entrepris de reproduire cette partie en PHP. Cependant, l'utilisation de packages Composer existants a été entravée par des erreurs de décodage et une incompatibilité avec les formats MRZ spécifiques, rendant ces solutions inopérantes.

Création d'un Algorithme Personnalisé de Décodage :

- Face à l'impasse avec les packages existants, j'ai entrepris de créer mon propre algorithme de décodage de bande MRZ. Cette initiative a demandé une analyse approfondie des spécificités des MRZ, en mettant l'accent sur les cartes d'identité principalement, avec des extensions futures pour les passeports et les permis de conduire.

Adaptation à Divers Formats de MRZ :

- L'algorithme personnalisé a dû être conçu de manière à être adaptable à divers formats de MRZ, tenant compte des particularités des cartes d'identité, des passeports et des permis de conduire.

La réalisation du décryptage des MRZ a posé des défis. Initialement, le décodage en TypeScript était efficace mais limité au client, entraînant une adaptation stratégique pour fonctionner localement sans connexion internet. Les problèmes avec les packages PHP ont conduit à la création d'un algorithme personnalisé. Ces défis ont été des opportunités pour des solutions innovantes. La prochaine étape consiste à intégrer ces solutions pour assurer la fiabilité du processus

Conclusion

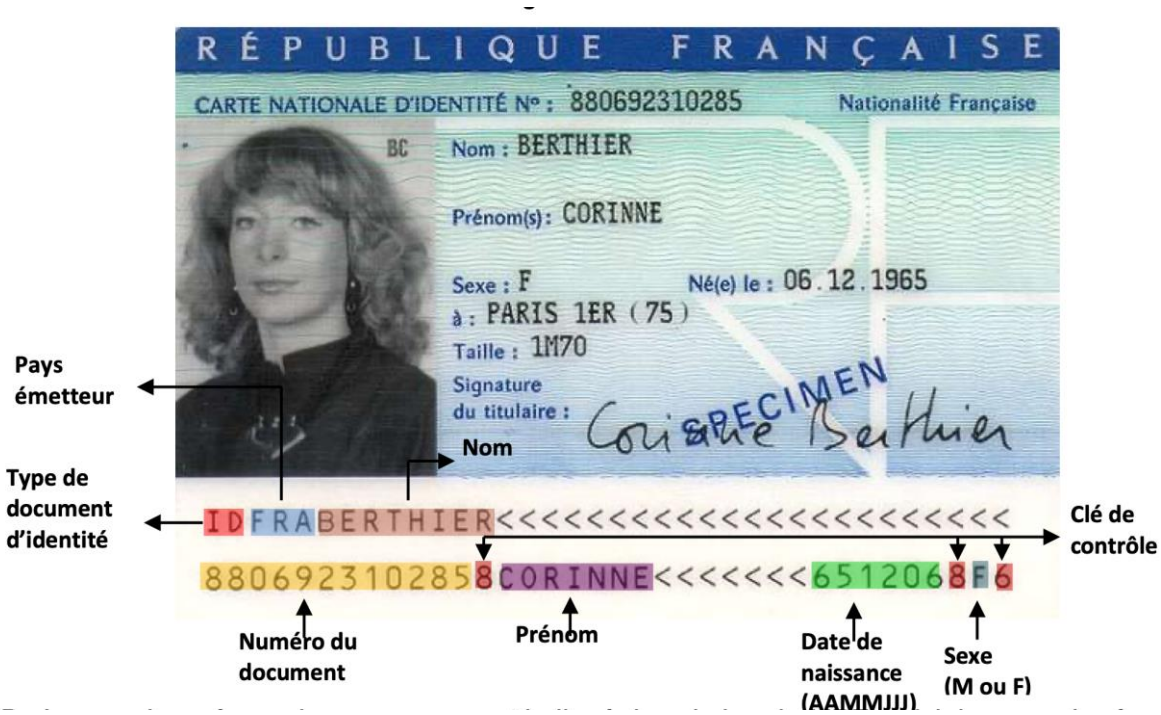
Le projet a connu des progrès significatifs, marquant des étapes clés dans son développement. Les maquettes de l'application ont été élaborées avec soin, offrant une représentation visuelle précise de l'interface à venir. Les pages de l'application ont été conçues et mises en œuvre de manière réussie, démontrant une avancée tangible dans la construction de l'interface utilisateur.

La fonctionnalité de la caméra, un élément central du projet a été intégrée, bien que des ajustements et des améliorations puissent encore être envisagés. L'acheminement efficace de la photo capturée vers le serveur, suivi de la récupération de cette photo pour la lecture des bandes MRZ, représente une réalisation significative. De plus, la partie locale du décodage en TypeScript est opérationnelle, renforçant la robustesse de l'application même dans des conditions hors ligne.

En perspective, mes futurs plans pour le projet se concentrent sur la finalisation de l'algorithme de décodage des bandes MRZ en PHP et l’automatisation de la détection d’une nouvelle photo sur le serveur contenant une MRZ.

Annexe

1. Photographie représentative de la MRZ d'une carte d'identité :



2. Photographie représentant le décodage de la MRZ en TypeScript (avec « mrz ») :

```typescript
import * as mrz_decrypt from "mrz";

export class DecryptMRZ{
    protected _mrz: Array<string>;
    protected _data: any;

    public constructor(mrz: Array<string>){
        this._mrz = mrz;
        this._data = mrz_decrypt.parse(this._mrz);
        this._data = JSON.stringify(this._data);
        this._data = JSON.parse(this._data);
    }

    public decryptMRZ(): Map<string, string>{
        let info_perso: Map<string, string> = new Map();
        let format = this._data.format;
        let issuing_state = this._data.details[1].value;
        let administrative_code = this._data.details[3].value;
        let issue_date = this.dateToSqlDate(this._data.details[4].value).toISOString().split("T")[0];
        let nom = this._data.fields.lastName;
        let prenom = this._data.fields.firstName;
        let birth = this.dateToSqlDate(this._data.fields.birthDate).toISOString().split("T")[0];
        let sex = this._data.fields.sex;

        info_perso.set("format", format);
        info_perso.set("issuing_state", issuing_state);
        info_perso.set("administrative_code", administrative_code);
        info_perso.set("surname", nom);
        info_perso.set("name", prenom);
        info_perso.set("birth", birth);
        info_perso.set("sex", sex);
        info_perso.set("issue_date", issue_date);

        return info_perso;
    }
```

3. Images illustrant l'exécution du programme pour une carte d'identité et un passeport :

```
const client = new Client(["IDFRAGONTIER<<<<<CLLLCLLLLC<<<343108",
                           "1108343010746MARION<<MELANI8906027F4"]);

const client1 = new Client(['I<UTOD23145890<1233<<<<<<<<<<<',
                            '7408122F1204159UTO<<<<<<<<<<<6',
                            'ERIKSSON<<ANNA<MARIA<<<<<<<<<<']);
```

id	format	issuing_state	administrative_code	surname	name	birth	sex	issue_date
1	FRENCH_NATIONAL_ID	FRA	343108	GONTIER CLLLCLLLLC	MARION MELANI	1989-06-02	female	2011-08-31
2	TD1	null	3	ERIKSSON	ANNA MARIA	1974-08-12	female	2014-09-30

4. Photographie représentant le passage de la photo à la récupération du type du document :

```php
use thiagoalessio\TesseractOCR\TesseractOCR;

class decryptMRZ{
    private $_scanMRZ;
    private $_tesseract;
    private $_imageMRZ;
    private $_text;
    private $array_infos;

    public function __construct($imageMRZ){
        $this->_scanMRZ = new ScanMRZ();
        $this->_imageMRZ = $imageMRZ;
    }

    private function ImageToText(){
        $this->_tesseract = new TesseractOCR($this->_imageMRZ);
        $this->_text = $this->_tesseract->run();
        $this->_text = str_replace([" ", "\n", "%"], "", $this->_text);
    }

    public function TextToMRZ(){
        $this->ImageToText();
        $this->_array_infos = $this->_scanMRZ->parseMRZ($this->_text);
        print_r($this->_array_infos);
    }

    public function decryptInfos(){
        $this->ImageToText();
        $lenght = strlen($this->_text);
        $id = "";
        for($i = 0; $i < $lenght; $i++){
            if($i < 2) $id .= $this->_text[$i];
        }
        echo $id;
    }
```